

Day 0 - Before You Write Any Code: The Absolute Basics

Day 0 — Before You Write Any Code: The Absolute Basics

This day exists so that Day 1 doesn't feel overwhelming. If you've already worked through the Python track's Day 0 primer, some of this will repeat (what a terminal is, what an error message means) — that's intentional, because this lesson is written to stand completely on its own. If any part is already familiar, skim it and move on.

What is JavaScript, and why is it unusual among programming languages?

JavaScript (often abbreviated **JS**) is a programming language — a way of writing instructions that a computer follows exactly, step by step (see the Python primer's "what is a program" section for the full version of this idea, if you want it — the short version: code is a list of instructions, and the computer does exactly what it says, not what you meant).

What makes JavaScript unusual is *where* it runs. Most languages run in one place — a program you install and execute. JavaScript runs in **two** very different places:

- 1 **Inside a web browser** (Chrome, Firefox, Edge, Safari) — this is JavaScript's original home. Every web page you've ever visited that does something interactive (a button that reacts when clicked, a form that checks your input before submitting, content that updates without reloading the page) is running JavaScript, right there in your browser, on your own computer.
- 2 **On a server, or directly on your computer, outside any browser at all** — using a program called **Node.js** (or just "Node"), which lets you run JavaScript exactly like you'd run a Python script, from your terminal, with no browser involved whatsoever.

This course teaches you the JavaScript *language* itself first, running it with Node — the exact same language rules apply whether the code eventually ends up running in a browser or in Node, so everything you learn transfers directly. You'll return to browser-specific JavaScript (interacting with a web page) properly if/when you study the HTML & CSS track's more advanced material, since that requires knowing HTML and CSS first.

What is TypeScript, and how does it relate to JavaScript?

TypeScript (often abbreviated **TS**) is not a separate, competing language — it's JavaScript, plus an extra layer of optional rules that let you describe what *type* of value each variable and function is expected to hold (a number, a piece of text, and so on), which a tool can then check *before* your code ever runs, catching a whole category of mistakes early. You'll spend Week 1 and most of Week 2 (Days 1-10) learning plain JavaScript, since TypeScript is built entirely on top of it and makes no sense without it — then Days 11-14 introduce TypeScript itself and have you use it for real.

Text editors and the terminal — same as any language

You'll write code in a plain-text code editor (VS Code is the most common choice, and happens to be made by the same company that created TypeScript, so it has excellent built-in support for both languages). If you already set up a terminal workflow for the Python track, you can reuse the exact same terminal window and editor here — nothing about your basic tools changes between languages. If this is genuinely your first time, see this same section in the Python track's [day00-primer/lesson.md](#) for a full explanation of what a text editor and a terminal actually are, since that explanation applies identically here.

A JavaScript file is a plain text file ending in `.js` (or `.ts` for TypeScript, starting Day 11) — for example, `hello.js`.

Running a JavaScript file with Node.js

Once you've saved a file called `hello.js` containing:

```
console.log("Hello, world!");
```

you run it from the terminal, in the same folder as the file, by typing:

```
node hello.js
```

Node reads `hello.js` top to bottom and runs each instruction. Here, that prints `Hello, world!` to the terminal.

`console.log(...)` is JavaScript's version of Python's `print(...)` — a built-in instruction (a "function," fully explained on Day 4) that displays whatever is inside its parentheses. You will use it constantly, especially early on, to see exactly what your code is doing at each step.

Notice the semicolon `;` at the end of that line. JavaScript uses semicolons to mark the end of a statement (roughly, "one complete instruction"), similar to how a period ends a sentence in English. JavaScript will often still work even if you forget one, thanks to a feature called "automatic semicolon insertion" that tries to guess where you meant to put them — but relying on that guessing is a common source of subtle, confusing bugs. **This course writes semicolons explicitly at the end of every statement, and you should too.**

The Node.js REPL — JavaScript's interactive shell

Just like Python has an interactive mode, typing `node` alone (with nothing after it) and pressing Enter drops you into Node's **REPL** (Read-Evaluate-Print Loop — see the Python primer for the full explanation of what this acronym means if you want it) — a mode where you type one line of JavaScript at a time and immediately see the result:

```
> 2 + 2
4
> console.log("hi")
hi
```

The `>` is Node's prompt — you don't type it yourself. To leave, type `.exit` and press Enter, or press Ctrl+C twice.

The browser console — JavaScript's OTHER interactive shell

Because JavaScript also runs inside web browsers, every modern browser has its own built-in REPL-like tool, called the **console**, part of a larger toolset called **DevTools** (Developer Tools). You open it by pressing F12 (or right-clicking a web page and choosing "Inspect"), then clicking the "Console" tab. You can type JavaScript directly in there too, and it runs immediately, exactly like Node's REPL — except it also has access to whatever web page is currently open. You won't need this until later, once you start working with actual web pages, but it's worth knowing this second REPL exists, since you'll see screenshots and instructions referencing it constantly in any JavaScript material you read online.

Reading a JavaScript error message

You will see error messages constantly — this is normal and expected, not a sign that you've broken something badly. Here's an example:

```
console.log("Hello"
```

Running this (notice the missing closing parenthesis) produces something like:

```
SyntaxError: missing ) after argument list
```

Just like Python, the error's category name (**SyntaxError**, and later you'll meet **TypeError**, **ReferenceError**, and others) tells you something specific: a **SyntaxError** means the problem is in how you **typed** the code — punctuation, structure — before the code even had a chance to run. A **TypeError** (which you'll meet properly Day 1) means the code ran, but tried to do something with a value that doesn't support that operation. Node will usually also show you a "stack trace" — several lines showing which file and line number the error happened at, and what called what, leading up to it. Just like the Python advice: **read the error message fully, starting with its category name and description, before touching your code**. Don't panic and start randomly changing things.

Curly braces `{}` — how JavaScript groups instructions together

Unlike Python, which uses indentation alone to show which lines belong together (see the Python primer if you're curious), JavaScript uses **curly braces** `{ }` to explicitly mark the start and end of a block of code that belongs together:

```
if (5 > 3) {  
    console.log("five is bigger");    // this line is INSIDE the if block  
    console.log("still inside");      // also inside  
}  
console.log("this always runs");      // OUTSIDE the if block -- always runs
```

Indentation (the spaces at the start of a line) is still used in JavaScript, but purely for **human** readability — unlike Python, changing the indentation alone, without touching the curly braces, does not change what the code actually does. The curly braces are what genuinely matter to JavaScript; the indentation is a courtesy to whoever reads the code (including future-you), and every editor will auto-indent for you as you type, matching the braces.

Comments — notes to humans that JavaScript ignores

```
// this is a single-line comment -- everything after // on this line is ignored

/* this is a
   multi-line comment --
   everything between /* and */ is ignored */
```

Exactly like Python's `#`, comments exist purely for a human reader — JavaScript skips them entirely when running your code.

A few words you'll see constantly, defined up front

- **Variable** — a name that refers to a value (full explanation Day 1).
- **Function** — a named, reusable block of instructions you can "call" (run) by name, optionally handing it some input (full explanation Day 4). `console.log(...)` is a function.
- **Argument** — a value you hand to a function when you call it. In `console.log("hi")`, `"hi"` is the argument.
- **String** — text data, written between quotes: `"hello"`, `'hello'`, or, as you'll see Day 5, using backticks: ``hello``.
- **Object** — in JavaScript, this word is used two ways, and it's worth flagging now so it doesn't confuse you later: in the general sense (like Python), it means "a piece of data" broadly. In the *specific* sense you'll meet on Day 3, `Object` refers to one particular kind of JavaScript value — a collection of named properties, similar to a Python dict. Context will make clear which meaning is intended.
- **Method** — a function that belongs to a specific piece of data, called with a dot, like `"hello".toUpperCase()`.
- **Bug** — a mistake in your code that makes it behave incorrectly. Not a moral failing — every programmer produces bugs constantly.
- **Debugging** — the process of finding and fixing a bug.

What "run the file and check the output" means for these exercises

Starting Day 1, every `exercises.js` file has spots marked `// TODO: implement`, similar in spirit to Python's `pass` placeholder — JavaScript's equivalent of "do nothing yet" is simply to leave a function body empty, or return `undefined` (JavaScript's version of Python's `None`, meaning "no value" — full explanation Day 1). Below the TODOs, each file has code that calls your functions with test inputs and prints whether each one is correct. Your loop for every day: write code → run `node exercises.js` → read the output → fix what's wrong → run again. This is, in essence, the daily rhythm of a working programmer at any experience level.

You're ready for Day 1

Go to `day01-basics/lesson.md` next.

Day 1 - Variables, Types, and Equality

Day 1 — Variables, Types, and Equality

If you haven't read [day00-primer/lesson.md](#) yet, read that first.

Objectives

- Declare variables correctly with `let`, `const`, and understand why `var` is avoided in modern code
- Meet JavaScript's core data types
- Understand `undefined` vs `null` — two different "nothing" values, which trips up every beginner at least once
- Understand the difference between `==` and `===`, and why professional JavaScript code almost always uses `===`
- Understand type coercion — JavaScript's tendency to automatically convert values between types, sometimes surprisingly

Declaring variables: `let`, `const`, and (avoid) `var`

```
let age = 25;           // a variable whose value CAN be reassigned later
const name = "Ana";     // a variable whose value CANNOT be reassigned after this line

age = 26;               // fine -- age was declared with let
name = "Bo";            // TypeError: Assignment to constant variable.
```

`let` declares a variable you intend to change later; `const` declares one you don't. **A strong, widely-followed convention in professional JavaScript: default to `const` for everything, and only use `let` when you specifically know you'll need to reassign the variable later.** This isn't just style — a `const` tells anyone reading your code (including future-you) "this value never changes after this point," which makes code easier to reason about; if you see `let`, you know to keep an eye out for where it gets reassigned.

You'll frequently see a third keyword, `var`, in older JavaScript code and tutorials. **Avoid it in new code.** `var` has looser, more confusing rules about which part of your code can see it (its "scope" — full treatment Day 4) that were considered a design mistake in hindsight; `let` and `const` (introduced later in the language's life, in a version called ES6/ES2015) fix those problems and are what all modern JavaScript uses. You should be able to recognize `var` when you see it in existing code, but never choose to write it yourself.

Important: `const` prevents reassigning the variable name, but does NOT make the value itself **unchangeable**, if that value is a mutable type (like the objects and arrays you'll meet Day 3):

```
const numbers = [1, 2, 3];
numbers.push(4);           // this is FINE -- we're not reassigning `numbers`, just modifying the array
console.log(numbers);      // [1, 2, 3, 4]

numbers = [9, 9, 9];       // TypeError -- THIS would be reassigning `numbers` itself, which is not allowed
```

This distinction — "the name can't be reassigned" versus "the value can't be changed" — will feel much more familiar once you've been through Day 1's mutability discussion in the Python track, if you've done that; if not,

don't worry, Day 3 of this track covers exactly this idea in depth once you've met arrays and objects properly.

The core data types

JavaScript calls its basic data types **primitives**. Check any value's type with the `typeof` operator:

```
typeof 42           // "number"
typeof 3.14         // "number" -- JavaScript has only ONE numeric type, unlike Python's separate
typeof "hello"      // "string"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof null         // "object" -- a famous, long-standing bug in JavaScript itself, exp
```

Type	Example	Notes
number	42, 3.14, -7	JavaScript has exactly one numeric type for everything — no separate "integer" type like Python's <code>int</code> . Internally it's always a floating-point number, which occasionally causes the same kind of rounding quirk you may have seen elsewhere: <code>0.1 + 0.2</code> is <code>0.30000000000000004</code> , not exactly <code>0.3</code> .
string	"hi", 'hi', `hi`	Text. Single quotes, double quotes, and backticks (template literals — full treatment Day 5) are all valid; be consistent within a project.
boolean	true, false	Yes/no. Lowercase, unlike Python's capitalized <code>True/False</code> — a very common typo for anyone switching between the two languages.
undefined	undefined	JavaScript's way of saying "this variable exists, but nothing has been assigned to it yet." You'll see this constantly, often unintentionally, once you start writing functions (Day 4).

<code>null</code>	<code>null</code>	JavaScript's way of saying "this variable deliberately, explicitly has no value" — a value a programmer chose to assign on purpose, as opposed to <code>undefined</code> , which usually means "nothing has been set yet."
<code>object</code>	<code>{}, []</code>	Everything else — collections of data, full treatment Day 3.

`undefined` vs `null` — two different flavors of "nothing"

This confuses every beginner (and plenty of experienced developers) at least once, so let's be precise: **`undefined` is what JavaScript gives you automatically** when something hasn't been given a value — a variable you declared but never assigned, a function parameter nobody passed a value for (Day 4), a property that doesn't exist on an object (Day 3). **`null` is a value a programmer deliberately assigns** to explicitly say "there is genuinely no value here, on purpose" — you will type `null` yourself; you will rarely if ever type `undefined` yourself.

```
let x;
console.log(x);           // undefined -- declared, but never given a value

let y = null;
console.log(y);           // null -- deliberately set to "nothing" by the programmer
```

A practical rule many teams follow: use `null` in your own code whenever you want to explicitly represent "no value" (e.g., "no user is currently logged in" might be `currentUser = null`), and treat `undefined` as something that shows up on its own, signaling "this wasn't set" rather than something you assign by hand.

`==` versus `===` — and why you should almost always use `===`

JavaScript has **two** different equality operators, and the difference between them is one of the most important things to get right as a beginner.

`===` ("strict equality") checks whether two values are equal **and** the same type — no conversion, no guessing:

```
5 === 5           // true
5 === "5"         // false -- different types (number vs string), so NOT equal
```

`==` ("loose equality") first tries to **convert** the two values to a matching type, and only then compares them — and its conversion rules are notoriously full of surprising special cases:

```
5 == "5"          // true -- "5" gets converted to the number 5 first, then compared
0 == false        // true -- false gets converted to 0
"" == false       // true
null == undefined // true -- these two DIFFERENT "nothing" values are considered == equal
null === undefined // false -- with strict equality, they're correctly treated as different
```

The near-universal professional convention: always use `===` and `!==` (strict "not equal"), never `==` or `!=`. The conversion rules behind `==` are inconsistent enough that even experienced developers can't always predict them without looking them up, and using `===` eliminates an entire category of subtle bugs before they can happen, by simply refusing to compare values of different types at all. You'll see `==` in older code and tutorials — recognize it, but don't write it yourself.

Type coercion — JavaScript's habit of automatically converting between types

Type coercion is what JavaScript does when an operation involves two different types — instead of raising an error the way Python often would, JavaScript tries to automatically convert one or both values so the operation can proceed, guided by its own internal rules. You already saw this above with `==`; it also happens with ordinary operators like `+`:

```
"5" + 3          // "53"  -- the number 3 is converted to the STRING "3", then the two strings are joined
5 + "3"          // "53"  -- same thing, other order
5 + 3            // 8      -- both numbers, ordinary addition, no coercion needed
"5" - 3          // 2      -- surprisingly, MINUS converts "5" to the number 5 first, then subtracts
```

Notice `+` and `-` behave completely differently when a string is involved: `+` treats a string as "please join text together," while `-` (which makes no sense for text) instead tries to convert the string to a number first. This inconsistency is exactly why relying on automatic coercion is considered risky in real code — **when you need a specific type, convert explicitly yourself**, rather than hoping JavaScript's automatic rules happen to do what you want:

```
Number("5")      // 5 -- explicit, deliberate conversion to a number
String(5)         // "5" -- explicit, deliberate conversion to a string
Boolean(0)        // false -- explicit, deliberate conversion to a boolean
```

Truthy and falsy — which values count as "true enough" in a condition

You'll meet `if` properly tomorrow (Day 2), but since it connects directly to today's types: every JavaScript value has an implicit true/false-ish answer when used somewhere expecting a boolean. The **falsy** values — everything that counts as "false-ish" — are: `false`, `0`, `""` (empty string), `null`, `undefined`, and `NaN` (a special "Not a Number" value you'll encounter when a math operation fails, e.g. `Number("abc")`). **Every other value is truthy**, including things that might surprise you at first: an empty array `[]` and an empty object `{}` are BOTH truthy in JavaScript — this is a genuine, important difference from Python, where an empty list/dict is falsy. Keep this specific difference in mind if you're coming from the Python track.

Exercises

Open `exercises.js`, implement each function where you see `// TODO`, and run it with `node exercises.js` to check your work against the printed PASS/FAIL output.

Day 2 - Control Flow: Conditionals, Loops, Truthy/Falsy

Day 2 — Control Flow: Conditionals, Loops, Truthy/Falsy

Objectives

- Use `if/else if/else` to make decisions
- Use `for`, `while`, and the two special JavaScript loops `for...of` and `for...in`, and know when each is right
- Use `switch` as an alternative to a long `if/else if` chain
- Use the ternary operator to pick between two values
- Solidify yesterday's truthy/falsy rules, now that you can actually use them in real conditions

`if` / `else if` / `else`

```
const age = 20;

if (age >= 18) {
  console.log("You can vote.");
} else {
  console.log("You cannot vote yet.");
}
```

Exactly like English: "if `age` is greater than or equal to 18, run the first block; otherwise, run the second." The curly braces `{ }` mark which lines belong to each branch (recall from Day 0: unlike Python, JavaScript uses braces, not indentation, to decide this — indentation here is purely for human readability).

Chain additional conditions with `else if`, checked in order, top to bottom, stopping at the first one that's true:

```
const score = 72;
let grade;

if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B";
} else if (score >= 70) {
  grade = "C";
} else {
  grade = "F";
}

console.log(grade); // "C"
```

Notice `grade` is declared with `let` above the `if`, then assigned inside it — this is a common pattern, since a variable declared with `const` *inside* one branch of an `if` wouldn't exist anymore once that branch's `{ }` block ends (you'll understand exactly why once you cover scope in depth on Day 4).

Comparison and logical operators

`===/!==` (Day 1), `>`, `<`, `>=`, `<=`. Combine conditions with `&&` (and), `||` (or), `!` (not):

```
if (age >= 18 && hasId) {
  console.log("allowed in");
}

if (isWeekend || isHoliday) {
  console.log("no work today");
}

if (!isRaining) {
  console.log("no umbrella needed");
}
```

Unlike Python, JavaScript does **not** support chained comparisons like `0 <= x < 10` the way you might expect — that expression is legal JavaScript, but doesn't do what you'd think (it evaluates `0 <= x` first, producing `true/false`, then compares THAT boolean to `10`, which is almost never what you meant). Always write chained range checks out fully and explicitly:

```
if (x >= 0 && x < 10) { // the CORRECT way to check a range in JavaScript
  console.log("x is between 0 and 9");
}
```

The ternary operator — an `if` that produces a value

```
const status = age >= 18 ? "adult" : "minor";
```

Read as: "the value is 'adult' IF `age >= 18`, otherwise (`:`) the value is 'minor'." Use this only when each branch is a single simple value; use a full `if/else` for anything more involved.

Loops

`for` — the classic counting loop

```
for (let i = 0; i < 5; i++) {
  console.log(i); // prints 0, 1, 2, 3, 4
}
```

A `for` loop has three parts, separated by semicolons, all inside the parentheses: **initialization** (`let i = 0` — runs once, before the loop starts), **condition** (`i < 5` — checked before every single pass; the loop stops the moment this becomes false), and **increment** (`i++` — runs after every pass through the loop body). `i++` is shorthand for `i = i + 1`; you'll also see `i += 1` (add-and-assign) used the same way.

Just like Python's `range()`, this starts at 0 and stops *before* reaching 5 — five iterations total (0, 1, 2, 3, 4), by convention, though unlike Python there's no separate `range` object here; you write the counting logic out explicitly yourself every time.

`while` — repeat until a condition becomes false

```
let count = 0;
while (count < 3) {
  console.log("still going:", count);
  count++;
}
```

```

}
console.log("done");

```

Use **while** when you don't know in advance exactly how many times you'll repeat something — you're waiting for some condition to change, rather than counting through a known range. **Forgetting to update the loop's condition variable inside the loop body creates an infinite loop** — the program will run forever. If your program seems to hang, this is the first thing to check.

`for...of` — looping over the VALUES in an array (or string)

```

const fruits = ["apple", "banana", "cherry"];

for (const fruit of fruits) {
  console.log(fruit);    // "apple", then "banana", then "cherry"
}

for (const character of "abc") {
  console.log(character); // "a", "b", "c"
}

```

This is JavaScript's closest equivalent to Python's `for item in some_list:` — walking through each **value** in an array, one at a time, in order. You'll use `for...of` far more often than the classic counting `for` loop above, once you're mainly working with collections of data (arrays, covered fully Day 3) rather than doing pure counting.

`for...in` — looping over the KEYS/property names of an object

```

const scores = { Ana: 90, Bo: 85 };

for (const name in scores) {
  console.log(name, scores[name]); // "Ana 90", then "Bo 85"
}

```

`for...in` gives you each property **name** (a string), not the value directly — you then use square brackets, `scores[name]`, to look up the actual value for that name. **Important, common point of confusion: `for...in` is for objects; `for...of` is for arrays (and strings, and other "iterable" collections).** Using `for...in` on an array technically works but is widely considered bad practice (it can include unexpected extra properties and doesn't guarantee numeric order) — always use `for...of` for arrays, and save `for...in` specifically for objects. You'll get much more comfortable with this distinction once you've properly met objects and arrays tomorrow (Day 3).

`break` and `continue`

Identical in spirit to Python's versions:

```

for (const number of [1, 2, 3, 4, 5]) {
  if (number === 3) {
    break;          // stop the loop entirely the instant we hit 3
  }
  console.log(number); // prints only 1, then 2
}

for (const number of [1, 2, 3, 4, 5]) {
  if (number === 3) {
    continue;       // skip JUST this one pass, keep looping
  }
}

```

```
    }  
    console.log(number);    // prints 1, 2, 4, 5 -- skips only 3  
}
```

`switch` — a multi-way branch for comparing one value against several exact possibilities

```
function describeDay(dayNumber) {  
  switch (dayNumber) {  
    case 1:  
      return "Monday";  
    case 2:  
      return "Tuesday";  
    case 6:  
    case 7:  
      return "Weekend!";    // falling through two cases with no break between them -- both 6 and 7  
    default:  
      return "Unknown day";  
  }  
}
```

`switch` checks `dayNumber` against each `case` value, top to bottom, and runs the code under the first match. `default` is the catch-all, equivalent to a final `else`. Notice `case 6:` has no code directly under it before `case 7:` — this is called "falling through," and it means both 6 and 7 end up running the exact same `return "Weekend!"` line. **A classic, very common switch bug**, in situations where you're not returning immediately like this example: forgetting a `break;` statement at the end of a case, which causes execution to keep running into the *next* case's code too, even though you only meant to match one — always double check whether you need an explicit `break;` (not needed here, since `return` already exits the function immediately, but you'll need it in cases that don't return).

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js` to check your work.

Day 3 - Data Structures: Arrays and Objects, Destructuring, Spread/Rest

Day 3 — Data Structures: Arrays and Objects, Destructuring, Spread/Rest

Objectives

- Get fluent with arrays: indexing, the core methods, and the crucial `map/filter/reduce` trio
- Get fluent with objects: JavaScript's key-value collection, roughly equivalent to Python's dict
- Understand destructuring — a compact way to pull values out of arrays/objects into variables
- Understand the spread (`...`) and rest (`...`) syntax — the same three dots, doing related but different jobs depending on context

Arrays — ordered collections

An array is written with square brackets, values separated by commas — visually identical to a Python list:

```
const fruits = ["apple", "banana", "cherry"];
```

Just like Python, items are indexed starting at 0:

```
fruits[0]      // "apple"
fruits[2]      // "cherry"
fruits.length  // 3 -- a PROPERTY (no parentheses), not a function call, unlike Python's len(fruits)
fruits[10]     // undefined -- NOT an error! Reading past the end just gives you undefined
```

This last point is a genuine, important difference from Python worth noting now: reading an out-of-range index in JavaScript does not raise an error the way Python's `IndexError` does — it silently gives you `undefined`. This means a bug from an out-of-range index can go unnoticed for longer in JavaScript, since nothing crashes immediately — you'll just get confusing `undefined` values showing up further along in your program. Get in the habit of checking `array.length` when the range is uncertain.

Core array methods

```
const nums = [3, 1, 2];

nums.push(4);           // adds to the END, returns the new length -- [3, 1, 2, 4]
nums.pop();             // removes AND returns the last item -- nums is now [3, 1, 2]
nums.unshift(0);        // adds to the START -- [0, 3, 1, 2]
nums.shift();           // removes AND returns the first item -- [3, 1, 2]

nums.includes(1);        // true -- does the array contain this value?
nums.indexOf(1);         // 1 -- the INDEX where 1 is found, or -1 if not found
nums.slice(0, 2);        // [3, 1] -- like Python slicing: start included, end excluded
nums.join(", ");         // "3, 1, 2" -- combine into one string, like Python's ' '.join(...)
[...nums].sort((a, b) => a - b); // sorted COPY -- see the spread section below for why
```

`.sort()` on its own has a famous gotcha: by default it sorts based on treating everything as TEXT, not numbers — `[10, 2, 1].sort()` gives you `[1, 10, 2]`, not `[1, 2, 10]`, because it compares the strings "10", "2", "1" alphabetically. To sort numbers correctly, you must pass a **comparator function**: `nums.sort((a, b) => a - b)` (ascending) — this is worth memorizing as a pattern now, even before Day 4 fully explains that `(a, b) => a - b` syntax (a small anonymous function, called an "arrow function").

`map`, `filter`, and `reduce` — the three most important array methods you'll use, constantly, for the rest of your career

These three methods are how idiomatic, modern JavaScript transforms and processes collections of data — you will see and use them daily in any real JavaScript job. Each one takes a small function as an argument (this is a preview of Day 4's "functions are values" idea) describing what to do to each item.

`.map()` — transform every item into something new, producing a brand new array of the same length:

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map((n) => n * 2);
console.log(doubled); // [2, 4, 6, 8]
```

Read `(n) => n * 2` as: "for each item, call it `n`, and produce `n * 2`." This is the direct equivalent of Python's list comprehension `[n * 2 for n in numbers]`.

`.filter()` — keep only the items that pass a test, producing a new (possibly shorter) array:

```
const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens); // [2, 4]
```

Equivalent to Python's `[n for n in numbers if n % 2 == 0]`.

`.reduce()` — combine every item down into a single result (a sum, a maximum, a single merged object — anything), by repeatedly applying a function that takes the "running total so far" and the "current item":

```
const total = numbers.reduce((sum, n) => sum + n, 0);
console.log(total); // 10
```

Read this carefully: `(sum, n) => sum + n` is a function of two things — `sum` (the running total so far) and `n` (the current item) — and the `0` at the very end is the **starting value** for `sum`, before the first item is even processed. `.reduce()` is genuinely the trickiest of the three to get comfortable with — don't worry if it takes a few tries in the exercises below to feel natural; it's worth the effort, since it's extremely common in real code (it's how you compute totals, group data, flatten nested arrays, and much more).

You'll use all three of these constantly from here on, often chained together:

```
numbers.filter(...).map(...).reduce(...).
```

Objects — key-value collections

An **object** is JavaScript's roughly-equivalent-to-Python's-dict — a collection of named properties, written with curly braces:

```
const person = {
  name: "Ana",
  age: 30,
  isActive: true,
};
```

```

person.name          // "Ana" -- "dot notation," the more common way to access a property
person["age"]        // 30 -- "bracket notation," required when the key is stored in a variable

person.email = "ana@example.com"; // adding a brand new property is just an ordinary assignment
person.age = 31;                // updating an existing property, same syntax
delete person.isActive;        // removes a property entirely

```

Use dot notation (`person.name`) whenever the key is a fixed, known name written directly in your code — it's shorter and clearer. Use bracket notation (`person[someVariable]`) when the key itself is stored in a variable, or when the key contains characters that wouldn't be valid as a plain identifier (like a space or a number at the start):

```

const key = "age";
console.log(person[key]); // 30 -- the variable `key` holds the STRING "age", used to look up the value

```

Looping over an object

```

for (const key in person) {
  console.log(key, person[key]);
}

Object.keys(person); // ["name", "age", "email"] -- an ARRAY of just the keys
Object.values(person); // ["Ana", 31, "ana@example.com"] -- an ARRAY of just the values
Object.entries(person); // [["name", "Ana"], ["age", 31], ...] -- an ARRAY of [key, value] pairs

```

`Object.entries()` combined with `for...of` is a common, clean way to get both key and value together, closely mirroring Python's `dict.items()`:

```

for (const [key, value] of Object.entries(person)) {
  console.log(key, value);
}

```

Checking whether a key exists

```

"name" in person // true -- checks whether the property exists at all, even if its value is undefined
person.name !== undefined // a common but slightly imperfect alternative -- fails if the value is null
person.hasOwnProperty("name") // true -- another common, explicit way to check

```

Destructuring — pulling values out into variables in one step

Destructuring lets you unpack values from an array or object directly into separate variables, in one concise line, instead of accessing each one individually:

```

// Array destructuring -- position-based
const coordinates = [10, 20];
const [x, y] = coordinates;
console.log(x, y); // 10 20

// Object destructuring -- NAME-based (must match the property names exactly)
const person = { name: "Ana", age: 30 };
const { name, age } = person;
console.log(name, age); // Ana 30

// You can rename while destructuring an object:

```

```
const { name: personName } = person;
console.log(personName);    // Ana

// And provide a default, used only if that property is missing/undefined:
const { country = "Unknown" } = person;
console.log(country);    // "Unknown" -- person had no `country` property at all
```

You'll see object destructuring constantly in function parameters starting Day 4 — it's an extremely common, idiomatic JavaScript pattern.

Spread (...) and rest (...) — the same three dots, two different jobs

This is a common point of confusion for beginners: ... looks identical in both uses, but does the *opposite* job depending on where it appears.

Spread — expands an existing array or object out into its individual elements, most often used to build a new array/object or to copy one:

```
const nums1 = [1, 2, 3];
const nums2 = [...nums1, 4, 5];    // [1, 2, 3, 4, 5] -- spreads nums1's items out, then adds more
const numsCopy = [...nums1];        // a genuine, independent COPY of nums1 -- NOT the same array

const person = { name: "Ana", age: 30 };
const updatedPerson = { ...person, age: 31 };    // { name: "Ana", age: 31 } -- copy person, then override
```

That last example is an extremely common real-world pattern: "make a new object just like this one, but with one field changed" — spreading first, then listing overrides afterward (later properties win over earlier ones with the same name).

Rest — the opposite operation: *gathers* multiple individual values back together into a single array, most often seen in function parameters (full treatment Day 4) or destructuring:

```
const [first, ...rest] = [1, 2, 3, 4];
console.log(first);    // 1
console.log(rest);    // [2, 3, 4] -- everything ELSE gets gathered into this array
```

A rough rule for telling them apart when you see ... in someone else's code: if it's spreading an *existing* collection out (usually on the right-hand side of an assignment, or inside a new array/object literal), it's spread. If it's gathering loose values *together* into one new array (usually on the left-hand side, in a destructuring pattern, or as a function parameter), it's rest.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.

Day 4 - Functions: Declarations, Arrow Functions, Scope, Closures, this

Day 4 — Functions: Declarations, Arrow Functions, Scope, Closures, `this`

Objectives

- Write functions three different ways: declarations, expressions, and arrow functions — and know when each is preferred
- Understand default parameters and rest parameters
- Understand scope (`let/const` block scope vs the older, looser `var` function scope)
- Understand closures
- Understand `this` — genuinely one of the trickiest ideas in JavaScript, and get a solid, correct mental model for it today

Three ways to write a function

Function declarations

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3)); // 5
```

This should look familiar from every example so far. A useful, JavaScript-specific quirk: function declarations are "hoisted" — meaning you can actually call one *before* the line where it's written in your file, because JavaScript processes all function declarations first, before running the rest of the code top to bottom:

```
console.log(greet()); // works fine, even though greet is "defined" below this line  
  
function greet() {  
  return "hi";  
}
```

Function expressions

```
const add = function (a, b) {  
  return a + b;  
};
```

Here, a function is created and immediately stored in a variable, `add` — this is called a **function expression** (as opposed to a declaration) because the function itself is just a value being assigned, exactly like assigning a number or a string. Unlike declarations, function expressions are **not** hoisted — you cannot call `add()` before this line runs.

Arrow functions — the modern, most common style for short functions

```
const add = (a, b) => {
  return a + b;
};

const addShort = (a, b) => a + b;    // "implicit return" -- no braces, no `return` keyword, just one expression

const square = (n) => n * n;         // one parameter -- parentheses are optional here, but keep them for clarity
const sayHi = () => console.log("hi"); // zero parameters -- parentheses ARE required
```

Arrow functions are the modern, preferred way to write short functions, especially ones passed as arguments to other functions (recall yesterday's `.map((n) => n * 2)`). When the function body is just one single expression, you can drop the curly braces and the `return` keyword entirely — the value of that one expression is automatically returned; this is called an **implicit return**, and it's an extremely common style you'll see everywhere in real JavaScript and TypeScript code.

When to use which: reach for arrow functions by default, especially for short, one-off functions (passed to `.map/.filter/.reduce`, event handlers, and similar). Use a plain `function` declaration for a "main," named function that represents a core, standalone piece of logic in your file — it reads slightly clearer at the top level, and gets the hoisting benefit above. There's one more, genuinely important difference between arrow functions and regular functions — how they handle `this` — covered later in this lesson, and it's actually the main technical reason arrow functions are often preferred, not just a style preference.

Default and rest parameters

```
function greet(name, greeting = "Hello") { // default value, used if the caller doesn't supply one
  console.log(`${greeting}, ${name}!`);
}

greet("Ana"); // "Hello, Ana!"
greet("Bo", "Good morning"); // "Good morning, Bo!"

function sum(...numbers) { // REST parameter -- gathers any number of arguments into one array
  return numbers.reduce((total, n) => total + n, 0);
}

console.log(sum(1, 2, 3, 4)); // 10 -- called with any number of arguments at all
```

A default parameter's value is computed fresh, at the moment the function is called, each time it's actually needed — unlike Python's infamous mutable-default-argument trap (if you've done the Python track), JavaScript recomputes a default expression on every call, so this specific bug doesn't exist here in the same way.

Scope — which parts of your code can see which variables

Variables declared with `let` or `const` are **block-scoped** — they only exist inside the nearest enclosing pair of curly braces `{ }`:

```
if (true) {
  const message = "hello";
  console.log(message); // "hello" -- fine, we're still inside the block
}
```

```
console.log(message); // ReferenceError: message is not defined -- outside the block, it d
```

This is one of the concrete reasons `let/const` are preferred over the older `var`, which is **function-scoped** instead — it leaks out of `if/for/while` blocks and is only contained by an enclosing `*function*`, which historically caused a lot of confusing bugs (a variable you thought was safely "local" to one small block turning out to still be visible much further away).

```
function calculate() {
  const result = 42;
  return result;
}

calculate();
console.log(result); // ReferenceError: result is not defined -- result only existed inside calcul
```

Closures — a function that remembers where it was created

Exactly the same underlying idea as the Python track's closures, if you've seen that — a function defined *inside* another function can "remember" variables from the outer function, even after the outer one has already finished running:

```
function makeMultiplier(factor) {
  return function (n) {
    return n * factor; // `factor` is remembered from the ENCLOSING function
  };
}

const double = makeMultiplier(2);
const triple = makeMultiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
```

Each call to `makeMultiplier` creates its own separate, private `factor`, which is why `double` and `triple` don't interfere with each other, despite being built from the same inner function shape. You'll rely on this exact mechanism constantly once you're writing more advanced JavaScript — it's how a huge amount of real-world JavaScript manages private, hidden state.

`this` — genuinely the trickiest thing in today's lesson, so read carefully

`this` is a special keyword that refers to "whatever object this function is currently being called as a method of" — but, crucially, **its value is decided by *how* a function is called, not by where the function was written.** This is different from most other languages' equivalent concept, and it's the single most common source of confusing JavaScript bugs for people learning the language.

```
const person = {
  name: "Ana",
  greet: function () {
    console.log(`Hi, I'm ${this.name}`);
  },
};
```

```
person.greet();    // "Hi, I'm Ana" -- `this` refers to `person`, because greet was called AS person
```

Here, `this` correctly refers to `person`, because the function was called using `person.greet()` — calling a function "through" an object like this is exactly what sets `this` to that object. But watch what happens if you take that same function out and call it differently:

```
const greetFunction = person.greet;
greetFunction();    // "Hi, I'm undefined" -- `this` is NOT person anymore!
```

`this` is no longer `person`, because this time the function was called plainly, on its own, with no object in front of the dot — so `this` doesn't refer to `person` at all anymore. **This exact scenario is extremely common in real bugs** — for instance, passing `person.greet` as a callback to something else (like an event handler, which you'll eventually meet) silently loses its connection to `person`.

Arrow functions handle `this` completely differently — and this is the real, practical reason they're preferred`

Arrow functions do **not** get their own `this` at all — instead, they simply use whatever `this` was already in the surrounding code where the arrow function was written (this is called "lexical `this`," meaning it's determined by *where the code is written*, not by *how it's later called* — the opposite of a regular function's `this`):

```
const person = {
  name: "Ana",
  hobbies: ["reading", "hiking"],
  printHobbies: function () {
    // regular function used here deliberately, so `this` correctly refers to `person` inside it
    this.hobbies.forEach((hobby) => {
      // arrow function here -- it has NO `this` of its own, so it uses `printHobbies`'s `this`
      console.log(`${this.name} enjoys ${hobby}`);
    });
  },
};

person.printHobbies();
// Ana enjoys reading
// Ana enjoys hiking
```

If that inner function had instead been written as a regular `function` (not an arrow function), `this` inside it would NOT be `person` — it would be `undefined` (in modern JavaScript's strict mode) or something else entirely unrelated, because `.forEach()` calls its given function plainly, without any object in front of it, exactly like the `greetFunction()` example above. This exact distinction — "does `this` refer to what I expect, or not" — is precisely why arrow functions became so popular for anything nested inside a method: they simply inherit the correct, expected `this` from their surroundings, with no surprises.

A practical rule of thumb for today: when defining a method directly on an object (or, starting tomorrow, inside a class), and it genuinely needs access to that object's own data via `this`, write it as a regular function (or, in classes, the normal method syntax you'll see tomorrow). When writing a smaller function *nested inside* that method — a callback passed to `.forEach/.map/similar` — prefer an arrow function, specifically so it correctly inherits the outer `this` rather than losing it.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.

Day 5 - Strings, Template Literals, JSON, and Dates

Day 5 — Strings, Template Literals, JSON, and Dates

Objectives

- Get comfortable with the string methods you'll use daily
- Use template literals fluently — you've already seen them a few times; today you get the full picture
- Use `JSON.stringify/JSON.parse` to convert between JavaScript objects and JSON text
- Get a working, practical understanding of the `Date` object

Strings — the methods that matter

Strings in JavaScript are immutable — exactly like Python strings, every "modifying" method actually returns a brand new string, leaving the original untouched:

```
const s = " Hello, World! ";

s.trim();           // "Hello, World!"           -- removes whitespace from both ends
s.toLowerCase();    // " hello, world! "
s.toUpperCase();     // " HELLO, WORLD! "
s.replace("l", "L"); // " HeLlo, World! " -- replaces only the FIRST match!
s.replaceAll("l", "L"); // " HeLLo, World! " -- replaces EVERY match
s.split(",");       // [" Hello", " World! "] -- splits into an ARRAY where
["a", "b", "c"].join(","); // "a,b,c" -- the reverse: glue an array of strings t
s.startsWith(" He"); // true
s.endsWith("! ");    // true
s.includes("World"); // true -- does this string CONTAIN that substr
s.indexOf("World");  // 9 -- the index where "World" starts, or
s.length;            // a PROPERTY, not a method -- no parenthesis
```

A genuine trap worth knowing about explicitly: `.replace()` with a plain string as its first argument replaces only the *first* match — you need `.replaceAll()` (a more recently added method) to replace every occurrence, or, in older code you might encounter, a special pattern-matching tool called a regular expression with a "global" flag. Always reach for `.replaceAll()` when you mean "every occurrence," rather than assuming `.replace()` does that.

Template literals — the modern way to build strings, using backticks

You've already used these in earlier lessons without a full explanation — a **template literal** is written with backticks (```) instead of regular quotes, and lets you embed the value of a variable or expression directly inside the text using `${...}`:

```
const name = "Ana";
const score = 91.5;
```

```

console.log(`${name} scored ${score}%`);           // "Ana scored 91.5%"
console.log(`${name} scored ${score.toFixed(1)}%`); // "Ana scored 91.5%" -- .toFixed(1) rounds

```

`${...}` can hold any JavaScript expression, not just a plain variable name — `${score.toFixed(1)}`, `${a + b}`, even a function call, all work directly inside the braces.

Template literals also let you write text across multiple lines directly, without any special escape character — something regular quoted strings can't do at all:

```

const multiline = `Line one
Line two
Line three`;

```

Template literals are the modern, preferred way to build any string that includes a variable's value — prefer them over the older style of joining strings with `+` (`"Hello, " + name + "!"`), which is harder to read, especially once several values are being combined.

Number formatting, since `.toFixed()` just came up

```

(3.14159).toFixed(2);           // "3.14" -- rounds to 2 decimal places, returns a STRING (not a number)
(1000000).toLocaleString();    // "1,000,000" -- inserts thousands separators, using your system's

```

Note `.toFixed()` returns a *string*, not a number — if you need to do further math with the rounded value, you'd have to convert it back with `Number(...)` first.

JSON — converting between JavaScript objects and portable text

JSON (JavaScript Object Notation) is, fittingly, named after this very language — it's a plain-text format for representing data (objects, arrays, strings, numbers, booleans, null) that's understood by virtually every programming language, and is the standard way programs send structured data to each other over the internet (you'll use this for real on Day 11, talking to a web API).

```

const data = { name: "Ana", tags: ["admin", "active"], active: true };

const jsonText = JSON.stringify(data);           // '{"name":"Ana","tags":["admin","active"]}'
const jsonTextPretty = JSON.stringify(data, null, 2); // same, but nicely indented with 2 spaces

const parsedBack = JSON.parse(jsonText);         // converts the JSON string back into a JS object
console.log(parsedBack.name);                     // "Ana"

```

Naming pattern to remember: `stringify` = JavaScript value → JSON text (turn it INTO a string). `parse` = JSON text → JavaScript value (read a string and turn it back into real data). The middle `null` and `2` arguments to `JSON.stringify` are optional — `null` here just means "no special filtering of which properties to include" (a feature you won't need yet), and `2` sets the indentation width for pretty-printing. If you just want the compact, one-line version, call `JSON.stringify(data)` with no extra arguments at all.

A trap worth knowing, mirroring the Python track's exact same warning if you've seen it: JSON has no concept of `undefined`, functions, or several other JavaScript-specific things. `JSON.stringify` will simply *omit* any object property whose value is `undefined` or a function, entirely silently, without any warning — worth remembering if data seems to be mysteriously missing after a round trip through JSON.

The `Date` object — working with dates and times

```
const now = new Date();           // the current date and time, right now, at the moment this line is executed
console.log(now);                 // e.g. 2026-07-01T14:32:10.123Z (an ISO-format text representation)

const specific = new Date(2026, 0, 15); // January 15, 2026 -- NOTE: months are 0-indexed! 0 = January
console.log(specific.getFullYear());    // 2026
console.log(specific.getMonth());        // 0 -- January, NOT 1! This trips up every JS beginner
console.log(specific.getDate());         // 15 -- the day of the month
console.log(specific.getDay());           // day of the WEEK (0 = Sunday, 6 = Saturday)

now.toISOString();                 // "2026-07-01T14:32:10.123Z" -- a standard, portable text format, great for APIs
now.getTime();                     // a plain number: milliseconds since January 1, 1970 -- useful for comparisons
```

The single most important gotcha to remember today: months are zero-indexed (January is **0**, December is **11**), while days of the month are NOT (the 1st of the month is **1**, as you'd expect) — this specific inconsistency has confused countless JavaScript developers, at every experience level, and is worth simply memorizing now rather than re-discovering through a bug later.

Comparing dates: since `.getTime()` gives you a plain number (milliseconds), you can subtract two dates to find the time between them:

```
const start = new Date(2026, 0, 1);
const end = new Date(2026, 0, 15);
const millisecondsBetween = end.getTime() - start.getTime();
const daysBetween = millisecondsBetween / (1000 * 60 * 60 * 24); // convert milliseconds -> seconds
console.log(daysBetween);    // 14
```

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.

Day 6 - Errors, Modules, npm and Packages

Day 6 — Errors, Modules, npm and Packages

Objectives

- Handle errors with `try/catch/finally`, and create your own custom error types
- Understand JavaScript's modern module system (`import/export`) and how it differs from the older CommonJS (`require`) style you'll also encounter
- Understand npm and `package.json` — how JavaScript projects declare and install their dependencies

`try` / `catch` / `finally``

```
try {
  const result = riskyOperation();
  console.log(result);
} catch (error) {
  console.log("Something went wrong:", error.message);
} finally {
  console.log("This always runs, error or not.");
}
```

`try` runs the code that might fail; the instant something inside it throws an error, execution jumps immediately to `catch`, skipping any remaining lines in the `try` block. `error` (you can name this variable anything, but `error` or `err` is conventional) is the actual error object, and `error.message` gives you its human-readable description. `finally` always runs afterward, whether or not an error happened — useful for cleanup code that must run either way.

Throwing your own errors

```
function withdraw(balance, amount) {
  if (amount > balance) {
    throw new Error(`Insufficient funds: have ${balance}, need ${amount}`);
  }
  return balance - amount;
}

try {
  withdraw(100, 500);
} catch (error) {
  console.log(error.message); // "Insufficient funds: have 100, need 500"
}
```

`throw new Error("message")` creates a new error object and immediately raises it — if nothing catches it, Node prints it and stops the program, exactly like Python's uncaught exceptions.

Custom error types

For errors specific to your own program's logic, define your own error class (full class syntax tomorrow, Day 8 — this specific pattern is simple enough to use today) by **extending** JavaScript's built-in **Error**:

```
class InsufficientFundsError extends Error {
  constructor(message) {
    super(message);          // pass the message up to the built-in Error's own setup
    this.name = "InsufficientFundsError"; // gives it a distinct name, shown in stack traces
  }
}

function withdraw(balance, amount) {
  if (amount > balance) {
    throw new InsufficientFundsError(`need ${amount}, have ${balance}`);
  }
  return balance - amount;
}

try {
  withdraw(100, 500);
} catch (error) {
  if (error instanceof InsufficientFundsError) {
    console.log("Please add funds to your account.");
  } else {
    throw error; // re-throw anything we didn't specifically expect -- don't silently swallow
  }
}
```

instanceof checks whether an object was created from a particular class (or one of its subclasses) — you'll get the full treatment of **class**, **extends**, and **instanceof** tomorrow and the day after; for today, just recognize this pattern: defining your own error type lets calling code distinguish "the specific error I expected" from "something else entirely went wrong," and handle each appropriately, rather than blindly catching and hiding everything.

Never write an empty **catch block that does nothing** — this is JavaScript's equivalent of Python's dangerous bare **except: pass**, and it's just as dangerous here: it silently swallows every possible error, including ones you never anticipated, leaving you with no clue later about what actually went wrong.

Modules — splitting code across multiple files

A **module** is any single JavaScript file, treated as a self-contained unit whose specific pieces (functions, variables, classes) can be shared with, or "exported" to, other files. Modern JavaScript uses **ES modules** (the **import/export** syntax) — this is what you should default to writing:

```
// mathUtils.js
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

export const PI = 3.14159;

// main.js
```

```
import { add, subtract, PI } from "./mathUtils.js";

console.log(add(2, 3)); // 5
```

export in front of a function, variable, or class marks it as available to other files; anything without **export** stays private to that one file. `import { add, subtract } from "./mathUtils.js"` pulls in exactly the named things you ask for, from the given file path (the `./` means "in this same folder").

You can also have one **default export** per file — useful when a module's whole purpose is to provide one main thing:

```
// logger.js
export default function log(message) {
  console.log(`[LOG] ${message}`);
}

// main.js
import log from "./logger.js"; // no curly braces for a DEFAULT import -- and you can name it anything
log("Hello"); // "[LOG] Hello"
```

CommonJS — the older module style you'll still see constantly

Plain Node.js scripts (like every `exercises.js/solutions.js` file you've written this week) traditionally use a different, older system called **CommonJS**, using `require` and `module.exports` instead of `import/export`:

```
// mathUtils.js (CommonJS style)
function add(a, b) {
  return a + b;
}
module.exports = { add };

// main.js (CommonJS style)
const { add } = require("./mathUtils.js");
console.log(add(2, 3)); // 5
```

This is exactly the pattern you've already been using all week, at the bottom of every `solutions.js` file (`module.exports = { ... }`) — now you know what it means and why it's there. **You should know both systems, because both are extremely common in real, existing codebases** (CommonJS in older Node projects and much existing tooling; ES modules in modern projects and virtually all browser-based code) — but for brand-new code, ES modules (`import/export`) is the modern standard, and what you'll use starting with TypeScript on Day 11.

npm and `package.json` — installing and managing other people's code

npm ("Node Package Manager") is how you install code that someone else has already written and published, so you don't have to write everything yourself from scratch. Every JavaScript/Node project has a `package.json` file at its root, describing the project and exactly which external packages it depends on:

```
npm init -y          # creates a starter package.json for a new project (the -y accepts all the defaults)
npm install lodash    # downloads the "lodash" package and adds it as a dependency in package.json
```

This creates (or updates) a file that looks roughly like:

```
{
```

```
    "name": "my-project",
    "version": "1.0.0",
    "dependencies": {
      "lodash": "^4.17.21"
    }
  }
}
```

It also creates a `node_modules/` folder, where the actual downloaded package code lives, and a `package-lock.json` file, which records the *exact* versions of everything installed (including packages your direct dependencies themselves depend on), so the install can be reproduced identically on another machine. `node_modules/` **should never be committed to git** — it's large, entirely derived from `package.json`, and gets recreated automatically by running `npm install` on any machine that already has the `package.json` and `package-lock.json` files (you'll set up a proper `.gitignore` for this on Day 13).

Once installed, you use a package exactly like any of your own modules:

```
const _ = require("lodash");           // CommonJS style
// or: import _ from "lodash";         // ES module style, if your project is configured for it

console.log(_.capitalize("hello"));    // "Hello"
```

You'll install and use your first real packages for real starting Day 11 (a testing tool, and eventually the TypeScript compiler itself).

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.

Day 7 - Mini Project: Node.js CLI Task Tracker (Week 1 Review)

Day 7 — Mini Project: Node.js CLI Task Tracker (Week 1 Review)

Objectives

Combine everything from Days 1-6 into one working program:

- variables/types/equality (Day 1)
- control flow (Day 2)
- arrays and objects as your data model (Day 3)
- functions, scope, closures (Day 4)
- strings, template literals, JSON (Day 5)
- error handling and modules (Day 6)

This is intentionally less hand-held than the previous days — the point of a review project is to make the design decisions yourself, using what you've learned, and get stuck in the productive way.

What's actually new today: command-line arguments in Node

You haven't used these yet. Node gives you access to whatever was typed after your script's filename via a special array, `process.argv`:

```
console.log(process.argv);
```

If you ran this as `node tasks.js add "Buy milk"`, you'd see something like:

```
["/path/to/node", "/path/to/tasks.js", "add", "Buy milk"]
```

Notice `process.argv[0]` is always the path to the `node` program itself, and `process.argv[1]` is always the path to your script — the arguments you actually care about start at index **2**, not index 1 (a small but easy-to-forget difference from what you might expect, and a common early bug). A common first step in any script like this is to slice those off:

```
const args = process.argv.slice(2); // ["add", "Buy milk"] -- just the REAL arguments, nothing else
```

The brief

Build a **command-line task tracker**, `tasks.js`, run repeatedly from the terminal, that manages a to-do list persisting between runs (saved as JSON — you covered `JSON.stringify/JSON.parse` on Day 5; today you'll additionally need to read and write actual files, using Node's built-in `fs` module, introduced just below).

Required features

- 1 `node tasks.js add "Buy milk"` — adds a new task, not yet complete
- 2 `node tasks.js list` — prints all tasks with an index number and a `[ ]/[x]` complete marker
- 3 `node tasks.js done 2` — marks task #2 as complete
- 4 `node tasks.js remove 2` — deletes task #2
- 5 Tasks persist in a file (e.g. `tasks.json`) between runs

Suggested data model

An array of objects:

```
[
  { description: "Buy milk", done: false },
  { description: "Walk dog", done: true },
]
```

Reading and writing files with Node's `fs` module

Node's standard library includes a module called `fs` ("file system") for reading and writing files — you haven't used this yet, so here's exactly what you need:

```
const fs = require("fs");

// writing (synchronously -- i.e., this line finishes completely before the next line runs)
fs.writeFileSync("tasks.json", JSON.stringify(tasks, null, 2));

// reading
const fileContents = fs.readFileSync("tasks.json", "utf-8");
const tasks = JSON.parse(fileContents);
```

The `Sync` in `writeFileSync/readFileSync` means "do this immediately, and don't move on to the next line until it's done" — the simplest way to work with files, and completely appropriate for a small script like this one. (Node also offers asynchronous, non-blocking versions of these — you'll understand exactly why that distinction matters once you cover asynchronous JavaScript on Day 10; for a small CLI tool like today's, the synchronous versions are simpler and entirely fine to use.)

You'll need to handle the case where `tasks.json` doesn't exist yet (the very first time the script ever runs) — `fs.readFileSync` throws an error if the file isn't there, so wrap it in `try/catch` (Day 6) and fall back to an empty array.

Suggested architecture (you decide the details)

- A function to load tasks from the JSON file (handling the file-not-existing-yet case without crashing)
- A function to save tasks back to the JSON file
- One function per command (`addTask`, `listTasks`, `completeTask`, `removeTask`)
- Parse `process.argv.slice(2)` by hand to figure out which command was requested and its arguments

Edge cases worth handling (this is where the real learning is)

- What happens if the user runs `list` before ever running `add`? (file doesn't exist yet)
- What happens if the user runs `done 99` but there are only 3 tasks? (index out of range — remember Day 3: JS won't crash on an out-of-range array index on its own, so you need an explicit check)
- What happens if the user runs `done abc` (not a number)?
- What happens if the user runs the script with no arguments, or an unrecognized command?

None of these should crash with an ugly, unhandled error — they should print a clear, human-readable message.

How to approach this (process, not just code)

- 1 Sketch the function signatures you'll need as comments first, before writing logic.
- 2 Build `add` and `list` first, test manually by running the script from the terminal, before adding `done/remove`.
- 3 Get the happy path fully working before handling edge cases.
- 4 Once it all works, deliberately try to break it (the edge cases above) and fix what breaks.

A note on scope

Don't reach for classes or anything from Week 2 — the whole point is proving you can build something real with *only* Week 1 material. You'll rebuild something similar with classes and TypeScript by Day 14.

What's provided

- `starter.js` — an empty scaffold with function signatures and comments to get you started (optional — you can also start from a blank file)
- `solution.js` — a full reference implementation. **Build your own first.**

Manual test checklist

Run these in order from your terminal in this folder:

```
node tasks.js add "Buy milk"
node tasks.js add "Walk dog"
node tasks.js list
node tasks.js done 1
node tasks.js list
node tasks.js remove 2
node tasks.js list
node tasks.js done 99
node tasks.js done abc
node tasks.js bogus-command
node tasks.js
```

Day 8 - OOP Fundamentals: Classes, Constructors, Encapsulation

Day 8 — OOP Fundamentals: Classes, Constructors, Encapsulation

Objectives

- Understand what a class is in JavaScript and how `class` syntax works
- Understand the constructor and how `this` behaves inside a class (now that you understand `this` in general from Day 4)
- Understand private fields (`#field`) and why encapsulation — hiding internal details — is useful
- Understand instance methods vs. data, and when a class is the right tool

A class is a blueprint; an instance is a real object built from it

Exactly the same core idea as any object-oriented language: a **class** is a blueprint describing what every object built from it will have; an **instance** is one actual object built from that blueprint.

```
class Dog {
  constructor(name, breed) {
    this.name = name;
    this.breed = breed;
  }

  bark() {
    return `${this.name} says Woof!`;
  }
}

const rex = new Dog("Rex", "Labrador");    // `new` creates a brand-new INSTANCE of the Dog class
const fido = new Dog("Fido", "Poodle");

console.log(rex.bark());    // Rex says Woof!
console.log(fido.bark());    // Fido says Woof!
```

The `new` keyword is required in JavaScript when creating an instance of a class — forgetting it (`Dog("Rex", "Labrador")` without `new`) causes an error, since `Dog` is specifically designed to be used as a class, not called as a plain function.

The constructor — JavaScript's version of Day 8's `__init__` (if you know Python) or just "setup logic"

constructor is a specially-named method that runs automatically the moment you create a new instance with **new**. Its job is to set up that instance's starting data:

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
}

const p = new Point(3, 4);
console.log(p.x, p.y); // 3 4
```

Inside the constructor (and inside every regular method on a class), **this** refers to the specific instance being created or operated on — exactly the "called as a method" rule from Day 4's **this** explanation, since **new Dog(...)/rex.bark()** are both, under the hood, calling their functions in a way that correctly sets **this**.

Methods

A method is simply a function defined directly inside the class body (notice: no **function** keyword needed, unlike a plain object's methods from Day 4):

```
class Counter {
  constructor() {
    this.count = 0;
  }

  increment() {
    this.count += 1;
  }

  reset() {
    this.count = 0;
  }
}

const c1 = new Counter();
const c2 = new Counter();
c1.increment();
c1.increment();
console.log(c1.count); // 2
console.log(c2.count); // 0 -- completely separate from c1
```

Each instance gets its own independent copy of whatever's set in the constructor (**this.count** here) — **c1** and **c2** never interfere with each other, exactly like the Python track's equivalent example, if you've seen it.

Private fields — hiding internal details with `#`

Sometimes you want a piece of data to belong to an instance, but to be genuinely inaccessible from *outside* the class entirely — not just "not intended to be touched," but actually blocked by the language itself.

JavaScript supports this with a **#** prefix:

```
class BankAccount {
  #balance; // declares a PRIVATE field -- only accessible from inside this class
```



```

    constructor(owner, initialBalance) {
        this.owner = owner;
        this.#balance = initialBalance;
    }

    deposit(amount) {
        this.#balance += amount;
    }

    getBalance() {
        return this.#balance;
    }
}

const account = new BankAccount("Ana", 100);
account.deposit(50);
console.log(account.getBalance());    // 150
console.log(account.#balance);        // SyntaxError -- #balance is not accessible from outside t

```

This idea — deliberately hiding a piece of data and only exposing controlled ways to read or change it (here, `getBalance()` and `deposit()`) — is called **encapsulation**. Why bother hiding `#balance` at all, instead of just using `this.balance` (no `#`, freely accessible)? Because it lets the class guarantee its own rules: for instance, you could add validation inside `deposit()` (reject negative amounts) with total confidence that `#balance` can *only* ever be changed through methods that enforce that rule — nothing outside the class has any way to reach in and set `#balance` directly, bypassing your checks. Without the `#`, any outside code could do `account.balance = -99999` directly, completely ignoring any rules your methods were trying to enforce.

Getters and setters — methods that look like plain property access

You can define special methods that run automatically when a property is **read** or **assigned**, using `get` and `set`, letting you add logic (like validation, or a computed value) while still allowing the natural-looking `account.balance` syntax rather than requiring `account.getBalance()`:

```

class BankAccount {
    #balance;

    constructor(initialBalance) {
        this.#balance = initialBalance;
    }

    get balance() {
        return this.#balance;
    }

    set balance(amount) {
        if (amount < 0) {
            throw new Error("Balance cannot be negative");
        }
        this.#balance = amount;
    }
}

const account = new BankAccount(100);

```

```
console.log(account.balance); // 150 -- reads like a plain property, but actually calls the getter
account.balance = 200;       // looks like plain assignment, but actually calls the setter method
account.balance = -50;       // throws an Error, thanks to the setter's validation
```

This is a nice piece of syntax sugar: callers write `account.balance` and `account.balance = 200` exactly as if `balance` were an ordinary, plain property — but behind the scenes, your `get/set` methods are quietly running, giving you the chance to compute a value or validate an assignment.

When should you actually reach for a class, versus a plain object or a plain function?

Same practical guidance as any object-oriented language: use a **plain function** when you're just computing something from inputs, with no state to remember. Use a **plain object** (Day 3) for a simple bundle of related data, with no enforced guarantees about its shape and no meaningful behavior attached. Use a **class** once you have data *and* behavior that clearly belong together, especially when you want to guarantee every instance has a consistent shape, or you specifically want to hide internal details behind a controlled interface (private fields plus getters/setters, as above).

Exercises

Open `exercises.js`, fill in each `// TODO` inside the class definitions provided, and run `node exercises.js`.

Day 9 - OOP Advanced: Inheritance, Prototypes, Static Members

Day 9 — OOP Advanced: Inheritance, Prototypes, Static Members

Objectives

- Use `extends` and `super` to build one class on top of another
- Understand polymorphism in JavaScript
- Understand what's actually happening underneath `class` — JavaScript's prototype system (this is genuinely JS-specific and worth knowing)
- Understand static methods/properties — things that belong to the class itself, not to any one instance

Inheritance with `extends` and `super`

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    throw new Error("Subclasses must implement speak()");
  }

  introduce() {
    return `I am ${this.name} and I say ${this.speak()}`;
  }
}

class Dog extends Animal {
  speak() {
    return "Woof";
  }
}

class Cat extends Animal {
  speak() {
    return "Meow";
  }
}

const rex = new Dog("Rex");
console.log(rex.introduce()); // I am Rex and I say Woof
```

`class Dog extends Animal` means `Dog` automatically gets everything `Animal` has (its constructor, `introduce()`), and only needs to define what's genuinely different about a `Dog` — here, just `speak()`. `Animal.speak()` deliberately `throws`, meaning "every subclass MUST provide its own version" — if a new

subclass forgot to define `speak()`, calling it would immediately, clearly fail, rather than silently doing something wrong.

Notice `introduce()` calls `this.speak()` without knowing in advance whether `this` will be a `Dog`, a `Cat`, or some future subclass — it automatically gets the correct version for whatever object it's actually attached to.

This is polymorphism ("many forms"): one piece of code behaving differently depending on the actual object it's working with, decided while the program runs, not fixed in advance.

`super` — reaching the parent class's version of something

If a subclass needs its own constructor but still wants the parent's setup logic to run too, call it explicitly with `super(...)`:

```
class Employee extends Animal { // deliberately silly example, purely to show the mechanism
  constructor(name, salary) {
    super(name);                // runs Animal's constructor with `name`
    this.salary = salary;
  }
}

const e = new Employee("Ana", 50000);
console.log(e.name, e.salary); // Ana 50000
```

A JavaScript-specific rule worth knowing: if a subclass defines its own constructor, it MUST call `super(...)` before it can use `this` at all — JavaScript enforces this and will throw an error if you try to use `this` before calling `super()`. This makes sense once you know why: the parent class's constructor is what actually finishes setting up the object in the first place; `this` isn't fully ready to use until that's happened.

`super.someMethod()` (not just `super(...)`) also lets you call the *parent's* version of a method you've overridden, from inside the overriding method — useful when you want to add behavior on top of the parent's, rather than fully replacing it.

What's actually happening underneath `class` — prototypes

This section is genuinely specific to JavaScript, and worth knowing because you'll encounter the word "prototype" constantly in JavaScript documentation and interview questions. `class` syntax (which you've been using all of today and yesterday) is, under the hood, a cleaner, more familiar-looking way of writing something JavaScript has always done differently from most other object-oriented languages: instead of classes being their own separate concept, every JavaScript object has a hidden internal link to another object, called its **prototype**, and when you access a property or method that doesn't exist directly on an object, JavaScript automatically looks it up on that object's prototype instead, and so on up a chain, until it finds it or runs out of prototypes to check (this is called the "prototype chain").

```
class Dog {
  bark() {
    return "Woof";
  }
}

const rex = new Dog();
console.log(rex.hasOwnProperty("bark")); // false -- bark is NOT directly on rex itself
console.log(Object.getPrototypeOf(rex) === Dog.prototype); // true -- rex's prototype is Dog.prototype
```

Every instance of `Dog` shares the exact same single `bark` method, living once on `Dog.prototype`, rather than each instance carrying its own separate copy — this is more memory-efficient, and it's the actual mechanism `class` syntax is quietly using for you the whole time. You don't need to manually work with prototypes yourself in modern JavaScript — `class/extends` is the clean, modern syntax for everything you'd otherwise have to do manually — but recognizing the word "prototype" and knowing roughly what it refers to (where shared methods actually live, and how JavaScript looks up properties that aren't found directly on an object) will make a lot of JavaScript documentation and error messages (like `TypeError: X.prototype.y is not a function`) make much more sense.

Static methods and properties — belonging to the class itself, not to any instance

Sometimes a piece of data or a function logically belongs to the *class as a whole*, rather than to any one specific instance — for this, use `static`:

```
class Circle {
  static PI = 3.14159;    // a STATIC property -- one single value, shared by the class itself, not by instances

  constructor(radius) {
    this.radius = radius;
  }

  area() {
    return Circle.PI * this.radius ** 2;    // accessed via the CLASS name, not `this`
  }

  static fromDiameter(diameter) {    // a STATIC method -- a kind of alternate way to construct a Circle
    return new Circle(diameter / 2);
  }
}

console.log(Circle.PI);                // 3.14159 -- accessed directly on the class, no instance
const c = Circle.fromDiameter(10);    // creates a Circle with radius 5, via the static method
console.log(c.area());                 // uses Circle.PI internally
```

You call a static member on the class itself (`Circle.PI`, `Circle.fromDiameter(...)`) — never on an instance (`c.PI` would be `undefined`, since `PI` doesn't belong to any particular instance). Static methods are commonly used exactly like `fromDiameter` above: as an alternative, named way to construct an instance, when a single constructor isn't expressive enough to cover every way you might want to create one.

Getters/setters revisited, briefly, in the context of inheritance

The `get/set` syntax from Day 8 works exactly the same way in a subclass, and can be overridden just like any other method — a subclass can provide its own version of a getter/setter its parent defined, following the exact same polymorphism rules as `speak()` above.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.

Day 10 - Asynchronous JavaScript: Callbacks, the Event Loop, Promises, async/await

Day 10 — Asynchronous JavaScript: Callbacks, the Event Loop, Promises, async/await

Objectives

- Understand *why* JavaScript needs a special way of handling things that take time (this is arguably the single most defining feature of the language, and the topic most beginners find hardest)
- Understand callbacks, and the problem with them ("callback hell") that motivated everything else in this lesson
- Understand Promises — the modern building block for asynchronous code
- Understand `async/await` — the clean, modern syntax built on top of Promises
- Understand basic error handling in asynchronous code

Why does JavaScript need "asynchronous" code at all?

JavaScript, in both the browser and in Node, runs on a **single thread** — meaning it can only actually execute one single instruction at a time, ever (unlike some languages that can genuinely run multiple things simultaneously). Now imagine your program needs to read a large file from disk, or fetch data from a server somewhere across the internet — these things can take a noticeable amount of time (milliseconds to seconds). If JavaScript simply *paused* everything and waited, your entire program (in a browser: the entire web page, completely unresponsive to clicks or scrolling) would freeze solid until that slow operation finished.

Asynchronous ("async" for short) programming is JavaScript's answer: instead of pausing everything to wait, you say "start this slow operation, and let me know when it's done — meanwhile, keep running everything else." The rest of your program keeps working normally, and your "let me know when it's done" code runs later, once the slow thing actually finishes.

The event loop — the mechanism that makes this possible

JavaScript's runtime (Node, or a browser) uses something called the **event loop** to manage this. In simplified terms: JavaScript keeps a queue of "things to do once something slow finishes." It runs your normal, synchronous code (top to bottom, one instruction at a time, exactly like everything you've written so far this week) until there's nothing left to do *right now* — then it checks that queue, and if something on it is ready (like a network request that just finished), it runs the corresponding "let me know when it's done" code. Then it checks the queue again. This cycle — run what's ready, check the queue, repeat — is the "loop" in "event loop." You don't need to manage this cycle yourself; you just need to understand that it exists, since it explains some genuinely surprising ordering behavior you'll see in the exercises below.

Callbacks — the original, older approach

The earliest way JavaScript handled "let me know when it's done" was a **callback**: a function you hand to the slow operation, which it calls for you once it's finished:

```
const fs = require("fs");

fs.readFile("data.txt", "utf-8", (error, contents) => {
  if (error) {
    console.log("Something went wrong:", error.message);
    return;
  }
  console.log(contents);
});

console.log("This line runs BEFORE the file's contents are printed!");
```

Read this carefully: `fs.readFile` starts reading the file and immediately returns, WITHOUT waiting — the callback function `((error, contents) => { ... })` is only actually called later, once the file has genuinely finished being read. This is why `"This line runs BEFORE..."` prints **before** the file's contents, even though it's written **after** the `readFile` call in your source code — this is the entire point of asynchronous code: your program doesn't stop and wait.

The problem: "callback hell"

Callbacks work, but they get unwieldy fast once you need to do several async things in sequence, where each one depends on the previous one finishing first:

```
getUser(userId, (user) => {
  getOrders(user.id, (orders) => {
    getOrderDetails(orders[0].id, (details) => {
      console.log(details);
      // and this nesting just keeps growing sideways with each additional step...
    });
  });
});
```

This deeply-nested, sideways-growing shape is genuinely known in the JavaScript community as **"callback hell"** — it's hard to read, hard to add proper error handling to at every level, and hard to modify safely. This exact problem is what motivated everything else in today's lesson.

Promises — a value representing "the result of something that hasn't finished yet"

A **Promise** is an object representing an operation that hasn't completed yet, but will (eventually) either **resolve** (succeed, with some resulting value) or **reject** (fail, with some error). Instead of nesting callbacks, you chain `.then()` (for success) and `.catch()` (for failure):

```
function delay(ms) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`Waited ${ms}ms`), ms);
  });
}

delay(1000)
  .then((message) => {
```



```

        console.log(message);          // "Waited 1000ms" -- runs AFTER 1000ms have passed
        return delay(500);             // returning a Promise from .then() lets you CHAIN to the next
    })
    .then((message) => {
        console.log(message);          // "Waited 500ms"
    })
    .catch((error) => {
        console.log("Something failed:", error.message); // runs if ANY step above rejected
    });

```

This flattens what would have been deeply-nested callbacks into a readable, top-to-bottom **chain** — each `.then()` runs only after the previous step finished successfully, and a single `.catch()` at the end can handle a failure from *any* step in the chain, rather than needing separate error-handling at every nested level.

`setTimeout(callback, ms)` (used inside `delay` above) is a built-in function that runs `callback` after roughly `ms` milliseconds have passed, without blocking anything else — it's one of the most basic building blocks of asynchronous JavaScript, and a handy way to simulate "something slow" in your own practice code, exactly as done here.

`async`/`await` — the modern, clean syntax built on top of Promises

`async/await` doesn't replace Promises — it's a way of *writing* Promise-based code that reads almost exactly like ordinary, synchronous, top-to-bottom code, hiding most of the `.then()` chaining:

```

async function run() {
    const message1 = await delay(1000); // PAUSES this function here until the Promise resolves, v
    console.log(message1);              // "Waited 1000ms"

    const message2 = await delay(500);
    console.log(message2);              // "Waited 500ms"
}

run();

```

Two rules to know: `await` **can only be used inside a function marked `async`** (you'll get a `SyntaxError` otherwise), and `await` **pauses only that one function**, waiting for the Promise to settle, while the rest of your program (anything not inside that specific paused function) keeps running completely normally in the meantime — this is the same underlying async behavior as the callback example above, just written in a much more readable, linear style.

Error handling with `async`/`await` — back to `try`/`catch`

Because `async/await` makes asynchronous code look synchronous, you can go back to using ordinary `try/catch` (Day 6) to handle failures, instead of `.catch()`:

```

async function run() {
    try {
        const message = await delay(1000);
        console.log(message);
    } catch (error) {
        console.log("Something failed:", error.message);
    }
}

```

This is generally considered the clearest, most maintainable way to write asynchronous JavaScript today, and it's what you should default to for any new code — you should still recognize raw `.then().catch()` chains when you see them in existing code, and understand Promises well enough to know what `async/await` is actually doing underneath, but prefer `async/await` for anything you write yourself from here on.

Running several async operations at once with `Promise.all`

If you have several independent async operations that don't depend on each other, awaiting them one at a time wastes time waiting for each to finish before even starting the next. `Promise.all` lets you start them all simultaneously and wait for every one to finish:

```
async function run() {
  const [message1, message2] = await Promise.all([delay(1000), delay(500)]);
  console.log(message1, message2); // both ready together, after roughly 1000ms total (the LONGER)
}
```

This is a genuinely important performance pattern: if three independent network requests each take about a second, awaiting them one after another takes roughly three seconds total; `Promise.all`-ing them takes roughly one second total, since they all run concurrently.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`. Since this involves real timing, some tests may take a second or two to finish — that's expected.

Day 11 - Working With Real Data, and an Introduction to TypeScript

Day 11 — Working With Real Data, and an Introduction to TypeScript

Objectives

- Use the `fetch` API to make real HTTP requests and work with JSON from a web API
- Understand npm packages in more depth, now that you have a real reason to install one
- Understand *why* TypeScript exists, and set up your first TypeScript file
- Learn TypeScript's basic type annotations, and how to compile TypeScript into plain JavaScript

`fetch` — making HTTP requests

You met JSON on Day 5. Today you'll fetch real JSON from across the internet. Both modern browsers and modern Node.js (v18+, which you have) include a built-in `fetch` function for making HTTP requests — no separate package needed:

```
async function getUser(id) {
  const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }

  const data = await response.json(); // parses the response body as JSON -- itself returns a Promise
  return data;
}
```

`fetch(url)` returns a Promise that resolves once the server has responded (note: it resolves even for error responses like 404 — `response.ok` is how you check whether the status code indicates success, similar in spirit to what you might do in other languages). `response.json()` reads and parses the response body, and is itself asynchronous (since reading the full response body can take a moment), so it needs its own `await` too.

Real code should always account for the request possibly failing entirely — no internet connection, a timeout, a DNS failure — using `try/catch` around the whole thing:

```
async function getUser(id) {
  try {
    const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
    if (!response.ok) {
      throw new Error(`Request failed with status ${response.status}`);
    }
    return await response.json();
  } catch (error) {
    console.log(`Could not fetch user: ${error.message}`);
  }
}
```

```
        return null;
    }
}
```

Why TypeScript exists

You've now spent ten full days seeing what plain JavaScript is capable of — and also seeing, repeatedly, how easy it is to accidentally misuse a value's type without JavaScript complaining until something goes wrong at runtime (Day 1's `typeof`/coercion surprises, Day 3's silent `undefined` from an out-of-range array index, and so on). **TypeScript** adds a layer of optional type annotations on top of ordinary JavaScript, plus a separate tool (the **TypeScript compiler**, `tsc`) that checks those annotations **before** your code ever runs, catching an entire category of mistakes ahead of time — the exact same problem, and the exact same solution, discussed in the Python track's Day 12 regarding Python's type hints, if you've done that track — except in TypeScript's case, the checking is a much more central, expected part of daily work, not an optional add-on.

Crucially, TypeScript code doesn't run directly — it gets **compiled** (translated) into plain JavaScript first, which is what actually runs in Node or a browser. This is a genuinely new idea if everything you've done so far has been "write code, run code directly" — today you'll see the extra compile step for the first time.

Setting up TypeScript

```
npm install --save-dev typescript
npx tsc --init
```

`npm install --save-dev typescript` installs the TypeScript compiler as a **dev dependency** — a package needed only for **developing** your project (compiling/checking your code), not needed by the finished, running program itself (recall this same "regular dependency vs. dev dependency" distinction from the Python track's Day 13, if you've done it, or just take it as a new, TypeScript-specific idea here). `npx` runs a locally-installed package's command-line tool without needing to install it globally first. `tsc --init` creates a `tsconfig.json` file — TypeScript's configuration file, telling the compiler things like which JavaScript version to target and which files to include; you'll look at this in more depth on Day 13.

A TypeScript file ends in `.ts` instead of `.js`. To compile and run one:

```
npx tsc hello.ts          # compiles hello.ts into hello.js
node hello.js             # runs the resulting plain JavaScript, exactly like any other Node script
```

In practice, most projects use a tool (like `ts-node`, or a bundler) to compile and run in one step during development — you'll set this up properly on Day 13; for today, the two-step compile-then-run process above is worth doing manually at least once, so you understand exactly what's happening underneath.

Basic type annotations

```
let age: number = 25;
let name: string = "Ana";
let isActive: boolean = true;

function add(a: number, b: number): number {
    return a + b;
}
```

The `: number`, `: string`, `: boolean` after a variable name (or after a function parameter, or after a function's closing parenthesis, describing its return type) are **type annotations** — they tell the TypeScript compiler exactly what type of value is expected there. Try deliberately breaking one to see the compiler catch it:

```
let age: number = "twenty-five"; // Type 'string' is not assignable to type 'number'.
```

This exact mistake would have run without complaint in plain JavaScript (dynamic typing, from Day 1) — TypeScript catches it immediately, at compile time, before the code ever runs.

Arrays and objects in TypeScript

```
let numbers: number[] = [1, 2, 3]; // an array of numbers
let names: string[] = ["Ana", "Bo"]; // an array of strings

function greet(person: { name: string; age: number }): string {
    return `${person.name} is ${person.age}`;
}
```

That last example annotates the *shape* of an object directly inline — but writing this out every time gets repetitive fast, which is exactly what tomorrow's lesson (interfaces) solves.

Type inference — you don't always need to write the type out

TypeScript is often smart enough to figure out (infer) a variable's type automatically from its initial value, without you writing an annotation at all:

```
let age = 25; // TypeScript infers this is a `number`, automatically -- no annotation needed
age = "twenty-five"; // still an error! TypeScript remembers the inferred type and enforces it
```

A widely-followed convention: let TypeScript infer types for simple local variables whose value is obvious from context (like `age = 25` above), and write explicit annotations mainly for function parameters and return types, and for anything whose type genuinely isn't obvious just from looking at the initial value.

``any`` — the type that turns off type-checking (avoid it)

TypeScript has an escape hatch, `any`, which tells the compiler "don't check this value's type at all — treat it like plain JavaScript":

```
let anything: any = 5;
anything = "now a string"; // no error -- `any` disables type-checking entirely for this variable
```

Avoid `any` whenever you can. Using it defeats the entire purpose of using TypeScript in the first place for that particular value — it's occasionally necessary (working with a genuinely dynamic, unpredictable piece of data, or gradually converting an old JavaScript codebase to TypeScript file by file), but reaching for `any` as a quick way to make a compiler error go away, rather than actually fixing the underlying type mismatch, is a habit worth avoiding from day one.

Exercises

This day's exercises are in TypeScript. Open `exercises.ts`, fill in each `// TODO`, then compile and run it:

```
npx tsc exercises.ts
node exercises.js
```

If the compiler reports type errors, that's exactly it doing its job — read the message, fix the actual code (not just the type annotation), and recompile.

Day 12 - TypeScript Deep Dive + Testing with Jest

Day 12 — TypeScript Deep Dive + Testing with Jest

Objectives

- Learn `interface` and `type` for describing object shapes, without repeating yourself
- Learn generics — writing code that works with multiple types while still being type-checked
- Understand union types and how TypeScript handles "this could be one of several types"
- Write real tests using Jest, the most widely-used JavaScript/TypeScript testing framework

`interface` — naming a shape, once, and reusing it everywhere

Yesterday you annotated an object's shape inline: `{ name: string; age: number }`. Once you use that same shape more than once, naming it with an `interface` avoids repeating yourself:

```
interface Person {
  name: string;
  age: number;
  email?: string;    // the ? marks this property as OPTIONAL -- it may or may not be present
}

function greet(person: Person): string {
  return `${person.name} is ${person.age}`;
}

const ana: Person = { name: "Ana", age: 30 };           // valid -- email is optional
const bo: Person = { name: "Bo", age: 25, email: "bo@example.com" }; // also valid
const bad: Person = { name: "Cy" };                    // ERROR -- missing required property
```

An `interface` describes the *shape* a value must have — which properties it needs, and what type each one must be — without describing any actual behavior (that's what a `class`, from Days 8-9, is for). Any object matching that shape can be used wherever the interface is expected, regardless of how it was created — this is sometimes called "structural typing": TypeScript cares about the *shape* of your data, not what you formally declared it to be.

`type` — a more general alternative to `interface`

```
type ID = string | number;    // a UNION type -- explained below
type Point = { x: number; y: number }; // this looks a lot like an interface!
```

For plain object shapes, `interface` and `type` are largely interchangeable, and you'll see both conventions in real codebases (a common guideline: prefer `interface` for object shapes, since it can be extended and gives clearer error messages; use `type` for anything else — unions, combinations of types, and shapes that aren't

plain objects at all).

Union types — "this could be one of several types"

```
function formatId(id: string | number): string {
    return `ID-${id}`;
}

formatId(42);           // fine -- 42 is a number
formatId("abc");        // fine -- "abc" is a string
formatId(true);         // ERROR -- boolean is not part of the union string | number
```

`string | number` means "accept EITHER a string OR a number, nothing else." This is TypeScript's precise, checked way of expressing something JavaScript does constantly and loosely (a function that happens to work with either type) — with the union type, the compiler ensures every code path only does things valid for both possibilities, or forces you to explicitly check which one you actually have first:

```
function double(value: string | number): string | number {
    if (typeof value === "number") {
        return value * 2;           // TypeScript KNOWS value is a number here, thanks to the typeof
    }
    return value + value;           // and here, TypeScript knows it must be a string
}
```

This `typeof value === "number"` check is called **narrowsing**: TypeScript tracks, line by line, which specific type within a union is actually possible at each point, based on checks like this — inside the `if` block, only `number` is possible; after it, only `string` remains. This is a genuinely elegant piece of TypeScript's design, worth appreciating once it clicks.

Generics — writing one function/type that works with many types, still fully checked

Imagine writing a function that just returns whatever it's given, unchanged — its logic doesn't care about the specific type at all:

```
function identity(value: any): any {    // works, but loses ALL type information -- `any` disables
    return value;
}

const result = identity(5);    // TypeScript thinks `result` is `any` -- it has NO idea it's actually
```

Generics solve this properly — they let a function (or a type, or a class) be written once, work with any type, and still have that specific type tracked and checked correctly:

```
function identity<T>(value: T): T {    // T is a GENERIC TYPE PARAMETER -- a placeholder for "whatever"
    return value;
}

const numberResult = identity(5);      // TypeScript correctly infers T is `number` here -- num
const stringResult = identity("hello"); // and here, T is `string` -- stringResult is typed as
```

Read `<T>` as "this function works with some type, call it `T`, to be determined by whatever's actually passed in each time it's called" — and crucially, TypeScript remembers exactly which type `T` was for each individual call,

keeping full type-safety, unlike the `any` version above which throws that information away entirely.

You've actually already used a built-in generic type without necessarily noticing: `Promise<string>` (or `Promise<any>`, from yesterday) — the `<...>` there is exactly this same generic syntax, saying "a Promise that will eventually resolve with specifically a string" (or whatever type is inside the angle brackets).

A common, very practical use of generics: a reusable data structure or utility function that shouldn't care what type of data it holds:

```
function firstElement<T>(array: T[]): T | undefined {
    return array[0];
}

firstElement([1, 2, 3]);           // typed as number | undefined
firstElement(["a", "b"]);          // typed as string | undefined
```

Setting up Jest for testing

```
npm install --save-dev jest ts-jest @types/jest @types/node
```

(This has already been installed for you at the root of this whole `typescript-javascript` project, so every day from here on can use it without reinstalling.) **Jest** is the most widely-used JavaScript/TypeScript testing framework — its role and mental model are the direct parallel of the Python track's `pytest`, if you've done that track: it automatically finds and runs your tests, and reports pass/fail with helpful detail on failures.

Jest looks for files ending in `.test.js` or `.test.ts` (or inside a `__tests__` folder). Inside them, you write tests using `test()` (or its alias `it()`) and `expect()`:

```
// mathUtils.test.ts
function add(a: number, b: number): number {
    return a + b;
}

test("adds two positive numbers", () => {
    expect(add(2, 3)).toBe(5);
});

test("adds negative numbers", () => {
    expect(add(-1, -1)).toBe(-2);
});
```

`expect(actualValue).toBe(expectedValue)` is Jest's core assertion — `.toBe()` checks strict equality (`===`), the same rule you learned Day 1. Jest has many other "matchers" beyond `.toBe()`: `.toEqual()` (checks deep equality of objects/arrays, since two different array objects with identical contents aren't `===` equal — recall Day 1's `is===` distinction, or the Python track's `id()===`, if you've seen either), `.toBeNull()`, `.toContain()` (does an array/string contain something), and `.toThrow()` (does calling a function throw an error):

```
function divide(a: number, b: number): number {
    if (b === 0) {
        throw new Error("cannot divide by zero");
    }
    return a / b;
}

test("divide by zero throws", () => {
```

```
    expect(() => divide(10, 0)).toThrow("cannot divide by zero");
  });
```

Notice `expect(() => divide(10, 0))` — you must wrap the risky call in an extra arrow function when testing for a thrown error; if you called `divide(10, 0)` directly inside `expect(...)`, it would throw immediately, before Jest ever gets a chance to catch and check it properly.

Testing async code

```
test("delay resolves with the given value", async () => {
  const result = await delay(10, "hello");
  expect(result).toBe("hello");
});
```

Since Jest test functions can themselves be `async` (exactly like any other function, from yesterday's lesson), testing Promise-based/async code is straightforward — just `await` inside the test, exactly as you would anywhere else.

Run your tests with:

```
npx jest
npx jest --verbose      # shows every individual test's name, not just a summary
```

Exercises

Open `calculator.ts` and read it — it has type annotations already in place. Then open `calculator.test.ts`, fill in each `// TODO` test, and run `npx jest` from this folder.

Day 13 - Practical: Project Structure, npm Scripts, Linting, Git Workflow

Day 13 — Practical: Project Structure, npm Scripts, Linting, Git Workflow

Objectives

- Learn the conventional way a real TypeScript/JavaScript project is organized
- Understand `package.json` in depth, and npm scripts
- Understand `tsconfig.json`'s most important settings
- Set up ESLint and Prettier — the standard linting/formatting tools for this ecosystem
- Practice a real git workflow, applied to a TypeScript project

A conventional project layout

```
my-project/
  ■■■ node_modules/           # installed packages (Day 6) -- NEVER commit this to git
  ■■■ .gitignore
  ■■■ package.json
  ■■■ package-lock.json
  ■■■ tsconfig.json
  ■■■ .eslintrc.json (or eslint.config.js)
  ■■■ .prettierrc
  ■■■ README.md
  ■■■ src/
  ■   ■■■ index.ts
  ■   ■■■ utils.ts
  ■■■ tests/
  ■■■   ■■■ utils.test.ts
```

This should look familiar if you've seen the Python track's Day 13 "src layout" — the same underlying reasoning applies: keeping your actual source code inside `src/`, separate from configuration files sitting at the project root, keeps a growing project organized and matches what you'll find in essentially every real TypeScript project you open at a job.

`package.json` in depth

You've used `npm init` and `npm install` already (Day 6). Here's a more complete `package.json`:

```
{
  "name": "my-project",
  "version": "1.0.0",
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "test": "jest",
```

```

    "lint": "eslint src"
  },
  "dependencies": {
    "express": "^4.18.0"
  },
  "devDependencies": {
    "typescript": "^5.0.0",
    "jest": "^29.0.0",
    "eslint": "^9.0.0"
  }
}

```

`dependencies` are packages your **finished, running** program needs (like a web framework such as `express`); `devDependencies` are packages needed only for **developing** it (the TypeScript compiler, Jest, ESLint) — this exact distinction mirrors the Python track's Day 13 `dependencies/optional-dependencies.dev` split, if you've seen it, and the reasoning is identical: someone just **running** your finished project doesn't need your testing or linting tools installed alongside it.

npm scripts — named shortcuts for common commands

The `"scripts"` section defines shortcuts you run with `npm run <name>` (a couple of very common ones, `start` and `test`, can be run without the word `run`: just `npm start` / `npm test`):

```

npm run build      # runs "tsc" -- compiles your TypeScript
npm start          # runs "node dist/index.js" -- runs the compiled output
npm test           # runs "jest" -- runs your test suite
npm run lint       # runs "eslint src" -- checks your code for problems

```

This is genuinely important, practical convention: instead of every developer on a team needing to remember (or look up) the exact underlying command for building/testing/linting a specific project, everyone just runs `npm run build`, `npm test`, and so on — the **specific** underlying tool and its exact flags are hidden behind these consistent, memorable names, defined once in `package.json`.

`tsconfig.json`'s most important settings

You generated one on Day 11/12 with `npx tsc --init`. A few settings worth understanding, since you'll see them in every real TypeScript project:

```

{
  "compilerOptions": {
    "target": "ES2020",           // which JavaScript version to compile DOWN to (older = more compat.)
    "module": "CommonJS",        // which module system to compile to (CommonJS for Node, "ESNext" for browsers)
    "strict": true,              // turns on ALL of TypeScript's strictest checks -- always enable!
    "outDir": "./dist",          // where compiled .js files should be written, keeping track of source
    "rootDir": "./src"           // where your TypeScript source files live
  }
}

```

`"strict": true` **deserves special attention: always turn this on for any new project.** Among other things, it enables `strictNullChecks`, which forces you to explicitly handle the possibility of `null/undefined` wherever they might occur (recall Day 1's `null/undefined` distinction) — without it, TypeScript is considerably more permissive and misses a meaningful category of the exact bugs it exists to catch in the first place. Every professional TypeScript project you'll encounter enables `strict` mode; there's no good reason to leave it off for new code.

ESLint and Prettier — the standard linting/formatting tools

Just like the Python track's `ruff/black` (if you've seen that lesson), the JavaScript/TypeScript ecosystem has its own equivalent, widely-adopted pair of tools:

- **ESLint** — a **linter**: scans your code for likely bugs and bad patterns (an unused variable, a suspicious comparison, a Promise you forgot to `await`) without actually running it.
- **Prettier** — an **auto-formatter**: rewrites your code's formatting (indentation, quote style, line length) to one single, consistent style, ending any team debate about exactly how code should look, since the tool simply decides for everyone.

```
npm install --save-dev eslint prettier
npx eslint --init          # walks you through creating a starter ESLint config
npx eslint src             # check your code for problems
npx prettier --write src   # reformat your code automatically
```

Exactly like the Python track's equivalent tools, these are typically wired into a real project's **CI** (an automated system that runs checks every time code is pushed) and often into a "pre-commit hook" that runs automatically before a commit is even allowed to complete. Running them yourself, locally, before pushing, saves the frustrating cycle of pushing code and having CI reject it over a purely cosmetic issue you could have caught yourself in seconds.

The real git workflow, applied to a TypeScript project

This section is identical in spirit to the Python track's Day 13 (if you've done it) — the underlying discipline of version control doesn't change between languages, only which files you're tracking:

```
git init
git add .
git commit -m "Initial commit: project scaffold"

# ... after making changes ...
git status
git diff
git add src/
git commit -m "Add user validation logic"

git checkout -b feature/add-export-command
# ... work, commit ...
git push -u origin feature/add-export-command
```

****Commit messages should explain *why*, not restate *what* — `git diff` already shows the "what" in full detail.** Avoid committing directly to `main` on any team project; work through a branch and a reviewed pull request instead, exactly as covered in the Python track's Day 13, if you've seen it.

`.gitignore` for a TypeScript/JavaScript project

```
node_modules/
dist/
*.log
.env
coverage/
```

`node_modules/` (Day 6) is large and entirely recreated by `npm install` from `package.json/package-lock.json` — never commit it. `dist/` (or wherever your compiled JavaScript output goes) is a derived artifact from your TypeScript source, exactly like the Python track's `__pycache__/` — regenerated automatically, never hand-edited, so it doesn't belong in git either. `coverage/` is generated by test tools reporting how much of your code your tests actually exercise — also derived, also excluded.

Exercises

This is a hands-on, do-it-for-real day rather than an auto-graded one. Working inside this `day13-practical` folder:

- 1 Create a `src/greeter.ts` exporting a function `greet(name: string): string` returning ``Hello, ${name}!``.
- 2 Create a `tests/greeter.test.ts` with a Jest test for `greet` (run it with `npm run test` from the track root).
- 3 Write a `package.json` for this small folder (or reuse the shared root one) with `"build"`, `"test"` scripts.
- 4 Write a `.gitignore` covering `node_modules/`, `dist/`, `*.log`, `.env`, `coverage/`.
- 5 If you have git installed: `git init` here, then make two separate commits — one for the source, one for the tests plus `.gitignore` — each with a message explaining *why*.

A worked reference is in `solution/` if you get stuck on the mechanics — but run the actual git commands yourself.

Day 14 - Capstone Project: Expense Tracker (in TypeScript)

Day 14 — Capstone Project: Expense Tracker (in TypeScript)

Objectives

Build a complete, small application in TypeScript, using nearly everything from the last 13 days, mostly unassisted:

- OOP (Days 8-9): classes for `Expense` and `ExpenseTracker`, private fields, inheritance if you find a natural place for it
- Async (Day 10): even though this project doesn't strictly need network calls, structure at least one operation (loading/saving) as `async`, since real-world file/data operations very often are
- TypeScript (Days 11-12): interfaces, a generic utility if you find a natural place for one, strict type annotations throughout
- Errors (Day 6): custom error classes, clean handling of bad input
- Testing (Day 12): a real Jest test suite, written by you, not given to you
- Structure (Day 13): proper `package.json`, `tsconfig.json`, project layout

This is intentionally the least hand-held day. You get a specification, not a scaffold. Expect to get stuck — that's the design, not a bug in the lesson.

The brief

Build a CLI expense tracker that:

- 1 **Adds expenses:** `node dist/index.js add 42.50 "Groceries" --category food`
- Each expense has: amount (number), description (string), category (string, defaults to `"general"`), and a timestamp (use `new Date()` when created)
- 1 **Lists expenses**, optionally filtered by category: `node dist/index.js list [--category food]`
- 2 **Shows a summary:** `node dist/index.js summary` — total spent, and a breakdown of total per category
- 3 **Exports to CSV:** `node dist/index.js export expenses.csv` (a simple hand-written CSV writer is fine — you don't need a package for this)
- 4 **Persists between runs** as JSON, exactly like Day 7, now inside a proper class rather than free functions, and written in TypeScript with real interfaces

Required design elements (this is the point of the exercise — don't skip these to save time)

- An `Expense` class (Days 8-9) with `toJSON()`/a static `fromJSON()` method for converting to/from a plain object suitable for `JSON.stringify/JSON.parse` (recall: `Date` objects don't survive a JSON round-trip automatically — you'll need to convert explicitly, similar in spirit to the Python track's Day 14 `to_dict/from_dict` pattern, if you've seen it).
- An `ExpenseTracker` class owning an array of `Expense` objects, exposing methods like `addExpense`, `filterByCategory`, `total`, `totalByCategory`.
- A custom error class, `InvalidExpenseError` extends `Error`, thrown when amount is negative or description is empty — caught at the CLI layer and shown as a clean message, never a raw, unhandled crash.
- Full TypeScript typing throughout — every function parameter and return type annotated, an `interface` for the plain-JSON shape of an expense, `strict` mode enabled in `tsconfig.json`.
- A real `tests/` directory with Jest tests covering: adding a valid expense, adding an invalid expense (throws), filtering by category, total calculations, and CSV export producing the right rows.
- Proper project structure (Day 13): `package.json` with build/test scripts, `tsconfig.json`, `src/`, `tests/`, `.gitignore`, and if you have git, real commits with meaningful messages as you go.

Suggested (not mandatory) file layout

```

day14-capstone/
  package.json
  tsconfig.json
  .gitignore
  src/
    models.ts      # Expense, InvalidExpenseError
    tracker.ts     # ExpenseTracker
    storage.ts     # load/save JSON, export CSV
    index.ts       # CLI entry point
  tests/
    models.test.ts
    tracker.test.ts

```

How to approach 2+ hours of mostly-unassisted work without stalling out

- 1 **Write the `Expense` class first, in isolation**, and test it manually (a small scratch `.ts` file you compile and run, or the Node REPL after compiling) before touching the CLI or storage at all.
- 2 **Build `ExpenseTracker` next**, still testing manually — add a few expenses, call `.total()`, before wiring up JSON persistence.
- 3 **Add JSON save/load**, confirm round-tripping works, including the `Date` conversion problem mentioned above.
- 4 **Only then** wire up the CLI (`process.argv` parsing, exactly like Day 7).
- 5 **Write tests as you go**, not all at the end.
- 6 If you get stuck for more than ~15 minutes on one specific thing, that's a reasonable point to peek at the relevant part of `solution/` for just that piece — not the whole file.

A hint on the Date-through-JSON problem, since it's genuinely new today

`JSON.stringify` automatically converts a `Date` object into a string (via its `toISOString()` method) — but `JSON.parse` has no way of knowing that string was originally a `Date`, and hands you back a plain string instead. To persist and correctly restore an `Expense` with its timestamp, you need explicit conversion methods:

```
interface ExpenseData {
  amount: number;
  description: string;
  category: string;
  timestamp: string; // stored as an ISO string, not a Date
}

class Expense {
  // ... constructor, etc ...

  toJSON(): ExpenseData {
    return {
      amount: this.amount,
      description: this.description,
      category: this.category,
      timestamp: this.timestamp.toISOString(),
    };
  }

  static fromJSON(data: ExpenseData): Expense {
    return new Expense(data.amount, data.description, data.category, new Date(data.timestamp));
  }
}
```

Definition of done

- All 5 CLI commands work end to end, tested manually.
- `npm test` passes with no failures, and covers the required cases listed above.
- No raw, unhandled crashes reach the user for the invalid-input cases you're asked to handle.
- `npm tsc` compiles with no type errors, with `strict: true` enabled.

Reference solution

`solution/` has a complete working implementation. Give yourself a genuine attempt — ideally the full 2 hours — before opening it. The value of this whole two weeks culminates here: if you can build this mostly on your own, in TypeScript, you're at the level this track promised.